

Thread-to-Core Affinity

Academic year 2025-2026

Prof. Massimo Torquati
massimo.torquati@unipi.it

Outline

- Understanding CPU affinity concepts
- Simple examples
- Work-sharing threadPool with cooperative help and Worker affinity

Why thread placement matters?

- Modern multicore performance does not depend only on algorithmic parallelism and data structures
- It also depends on where threads run, whether they migrate across cores, and how well computation follows the memory hierarchy
- Thread migration may hurt locality, especially for long-lived worker threads
- On NUMA machines, placing computation far from the data may increase latency and reduce bandwidth
- Affinity is a way to constrain placement to improve locality and reduce run-to-run noise

CPU Affinity

- **CPU affinity** is an OS mechanism used to control where a thread or process is allowed to run
- Thread placement is often referred to as **thread mapping**, while **thread pinning** usually denotes a stronger placement constraint
- Binding can be:
 - **absent**, the thread may run on any allowed CPU
 - **preferential**, the runtime or scheduler tries to preserve locality
 - **hard**, the thread is restricted to a specific CPU set
- On Linux, affinity is expressed through an **affinity mask**, that is, the set of logical CPUs on which a thread or process may execute
 - the scheduler may move a thread only within the CPUs allowed by that mask

Motivations

- Better locality
 - Fewer migrations may keep caches warmer and reduce latency jitter
 - Better cache locality and TLB reuse
 - On NUMA systems, keeping threads close to their memory may reduce remote-memory traffic
- Less interference
 - Affinity may reduce unwanted sharing with unrelated threads
 - It can help avoid poor placements, for example on busy cores or SMT siblings
- More stable performance
 - Linux already tries to preserve some “natural” affinity
 - Explicit pinning is often useful in controlled experiments and low-jitter services

Affinity is more about reducing noise and preserving locality than about increasing peak throughput by itself

Inspecting CPU topology

- Pinning may hurt performance, especially with poor placements
 - Threads on the same physical core/SMT compete for resources
 - On NUMA systems, poor placement may increase remote-memory accesses
- We should know the core topology before choosing/trying CPU affinity
 - Sockets, NUMA nodes, physical and logical cores
- Inspecting CPUs' topology
 - numactl --hardware
 - lscpu
 - lstopo

```
[torquati@spmln ~]$ numactl --hardware
available: 2 nodes (0-1)
node 0 cpus: 0 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38
node 0 size: 63708 MB
node 0 free: 14842 MB
node 1 cpus: 1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39
node 1 size: 64457 MB
node 1 free: 497 MB
node distances:
node    0    1
  0:   10   21
  1:   21   10
[torquati@spmln ~]$
[torquati@spmln ~]$
[torquati@spmln ~]$ srun numactl --hardware
available: 2 nodes (0-1)
node 0 cpus: 0 1 2 3 4 5 6 7 16 17 18 19 20 21 22 23
node 0 size: 64316 MB
node 0 free: 62165 MB
node 1 cpus: 8 9 10 11 12 13 14 15 24 25 26 27 28 29 30 31
node 1 size: 64504 MB
node 1 free: 64041 MB
node distances:
node    0    1
  0:   10   21
  1:   21   10
```

Controlling affinity

- From the shell: **taskset** command. It restricts a process (or thread) to specific CPUs

- `taskset -c 0-2,8-10 ./myprogram`
 - run “myprogram” on CPUs 0,1,2,8,9, and 10
- `taskset -p <pid>`
 - show the current CPU affinity (affinity mask) of an existing PID
- `taskset -pc 2,4 <pid>`
 - set the CPU affinity of an existing PID
- `taskset -apc 2,4 <pid>`
 - apply the affinity setting to all threads of a process

- Programmatically on Linux:

- Standard C++ has no portable API for CPU affinity
- With POSIX threads, we can use `pthread_setaffinity_np`
- We can also query or set affinity with `sched_getaffinity/sched_setaffinity` system calls

In our examples, we will use the pthread-based interface to pin worker threads explicitly

Example

- The include file `Affinity.hpp` provides a few Linux-specific helper functions
 - `get_thread_affinity()`, query the current affinity mask
 - `pin_thread_to_core()`, bind the current thread to one logical CPU
 - `pin_thread_to_cores()`, bind the current thread to a CPU set
 - `get_current_cpu()`, report the CPU where the thread is currently running
- The file `affinity.cpp` example shows how to play with CPU affinity. It gets in input a CPU mask
 - `./affinity 0xC` // `0xC = 1100`, so the thread is allowed to run on CPU 2 and 3
- We can also restrict the process externally (it may require appropriate privileges)
 - `systemd-run --scope -p AllowedCPUs=8-15 ./affinity 0xC`
 - the effective affinity becomes the intersection of the external cpuset and the program request
 - Requested affinity is only part of the story. [The effective CPU set may be further restricted by the OS or by the execution environment, for example by SLURM](#)

Example code : *affinity.cpp*

Benchmarking affinity in an SPSC pattern

- One producer and one consumer communicate through an SPSC ring buffer
- Each thread can be pinned independently to a selected CPU
- After warmup, we measure steady-state throughput
- The goal is to compare good and bad placements
 - In this simple benchmark, when `prod-work = cons-work = 0`, placing producer and consumer on **SMT sibling cores** may help because communication dominates and the two threads stay very close in the cache hierarchy. This is not a general rule: as local computation increases, separate physical cores may become preferable

Usage example:

```
./spsc_affinity_bench --prod-cpu 0 --cons-cpu 1 --ring 4096 --prod-work 0 --cons-work 0 --duration 20 --warmup 3
```

Example code : `spsc_affinity_bench.cpp`

Work-Sharing threadPool with Affinity

- Two improvements to the `threadPool` implementation
 - Cooperative help by caller thread
 - Instead of waiting passively, the caller helps executing pending tasks
 - This reduces idle time and improves support for recursive task generation
 - Particularly useful when a task spawns subtasks and then waits for their completion
 - Worker affinity
 - Each Worker can be bound to a selected CPU at start-up
 - Binding is performed once, before entering the task-processing loop
 - This may improve cache locality and reduce scheduling noise

Example code : `include/threadPool.hpp`

Merge sort with a thread pool and affinity

- Recursive divide-and-conquer algorithm
- No recursive thread creation, subproblems become pool tasks
- One branch is submitted to the pool, the other is executed locally
- **Waiting is cooperative**, the caller may help the pool
- **Worker affinity** is managed directly by the pool
- Two variants:
 - Base: recursive task creation until the cutoff value is reached, then `std::stable_sort`
 - Max-depth: task creation only in the first recursion levels, then local subproblems are solved with `std::stable_sort`

Goal: show how a thread pool supports recursive task parallelism, especially when waiting is cooperative and task creation must be controlled

Example code : mergesort-pool.cpp and mergesort-pool-max_depth.cpp